```
!pip install squidpy scanpy anndata pandas matplotlib -q
```

```
  Preparing metadata (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 79.5/79.5 kB 4.0 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 91.2/91.2 kB 5.1 MB/s eta 0:00:00
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 87.8/87.8 kB 6.3 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 54.5/54.5 kB 3.6 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 193.9/193.9 kB 12.0 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.1/2.1 MB 43.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 174.3/174.3 kB 12.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.4/12.4 MB 87.3 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.1/60.1 kB 3.8 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 61.9/61.9 kB 4.8 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.5/1.5 MB 59.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 192.3/192.3 kB 13.4 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.2/23.2 MB 67.3 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 51.6/51.6 kB 3.3 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 44.7/44.7 kB 2.7 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 318.8/318.8 kB 14.6 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 85.0/85.0 kB 5.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.0/1.0 MB 37.6 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.2/9.2 MB 73.7 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 45.2/45.2 kB 2.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 83.5/83.5 kB 5.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13.2/13.2 MB 71.6 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.7/10.7 MB 81.9 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 43.3/43.3 kB 2.0 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 87.3/87.3 kB 6.5 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 88.0/88.0 kB 6.1 MB/s eta 0:00:00
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 14.3/14.3 MB 24.3 MB/s eta 0:00:00
  Building wheel for docrep (setup.py) ... done
  Building wheel for pims (setup.py) ... done
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 2.3.3 which is incompatible.
```

```
import os
import matplotlib.pyplot as plt

OUTPUT_DIR = "/content/outputs"
os.makedirs(OUTPUT_DIR, exist_ok=True)
_counter = [0]

def save_fig(name, fig=None):
    _counter[0] += 1
    path = f"{OUTPUT_DIR}/{_counter[0]:02d}_{name}.png"
    if fig is not None:
        fig.savefig(path, dpi=150, bbox_inches='tight')
    else:
        plt.savefig(path, dpi=150, bbox_inches='tight')
    print(f"✅ Saved: {os.path.basename(path)}")

print("✅ Output folder ready:", OUTPUT_DIR)
```

```
✅ Output folder ready: /content/outputs
```

```
# Prevent squidpy/scanpy from auto-showing figures
sc.settings.autoshow = False
```

```
%matplotlib inline

import pandas as pd
import matplotlib.pyplot as plt
import anndata as ad
import scanpy as sc
import squidpy as sq

sc.logging.print_header()
print(f"squidpy==={sq.__version__}")
```

```
squidpy==1.8.1
```

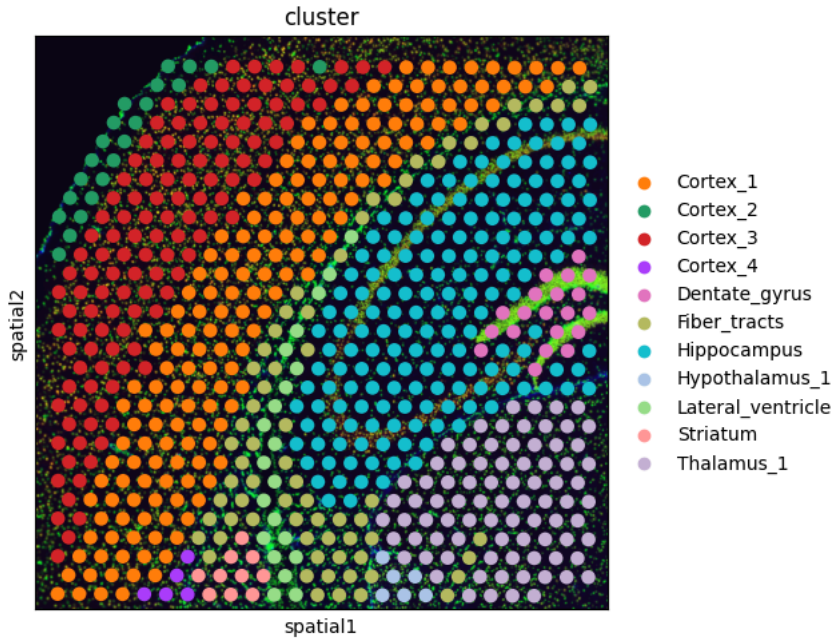
```
img = sq.datasets.visium_fluo_image_crop()
adata = sq.datasets.visium_fluo_adata_crop()
```

```
print("adata:", adata)
print("img:", img)
```

```
INFO     Downloading visium_fluo_image_crop.tiff from
         https://exampledata.scverse.org/squidpy/figshare/visium_fluo_image_crop.tiff
Downloading data from 'https://exampledata.scverse.org/squidpy/figshare/visium_fluo_image_crop.tiff' to file '/content/data/imag
100%
 317M/317M

INFO     Downloading visium_fluo_adata_crop.h5ad from
         https://exampledata.scverse.org/squidpy/figshare/visium_fluo_adata_crop.h5ad
Downloading data from 'https://exampledata.scverse.org/squidpy/figshare/visium_fluo_adata_crop.h5ad' to file '/content/data/annd
100%
 68.7M/68.7M

adata: AnnData object with n_obs × n_vars = 704 × 16562
    obs: 'in_tissue', 'array_row', 'array_col', 'n_genes_by_counts', 'log1p_n_genes_by_counts', 'total_counts', 'log1p_total_cou
    var: 'gene_ids', 'feature_types', 'genome', 'MT', 'n_cells_by_counts', 'mean_counts', 'log1p_mean_counts', 'pct_dropout_by_c
    uns: 'cluster_colors', 'hvg', 'leiden', 'leiden_colors', 'neighbors', 'pca', 'spatial', 'umap'
    obsm: 'X_pca', 'X_umap', 'spatial'
    varm: 'PCs'
    obsp: 'connectivities', 'distances'
img: ImageContainer[shape=(7272, 7272), layers=['image']]
```

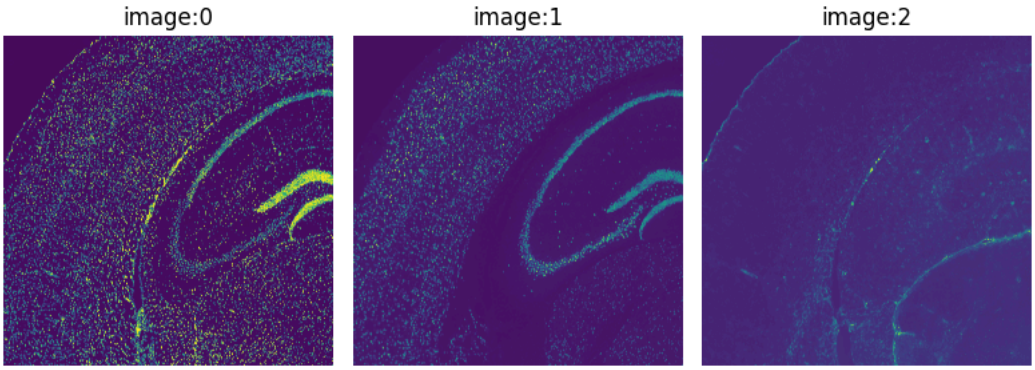
```
sq.pl.spatial_scatter(adata, color="cluster")
save_fig("spatial_scatter_clusters")
plt.show()
```

```
✅ Saved: 01_spatial_scatter_clusters.png
```




```
img.show(channelwise=True)
save_fig("fluorescence_channels_DAPI_NEUN_GFAP")
plt.show()
```

```
Saved: 02_fluorescence_channels_DAPI_NEUN_GFAP.png
```

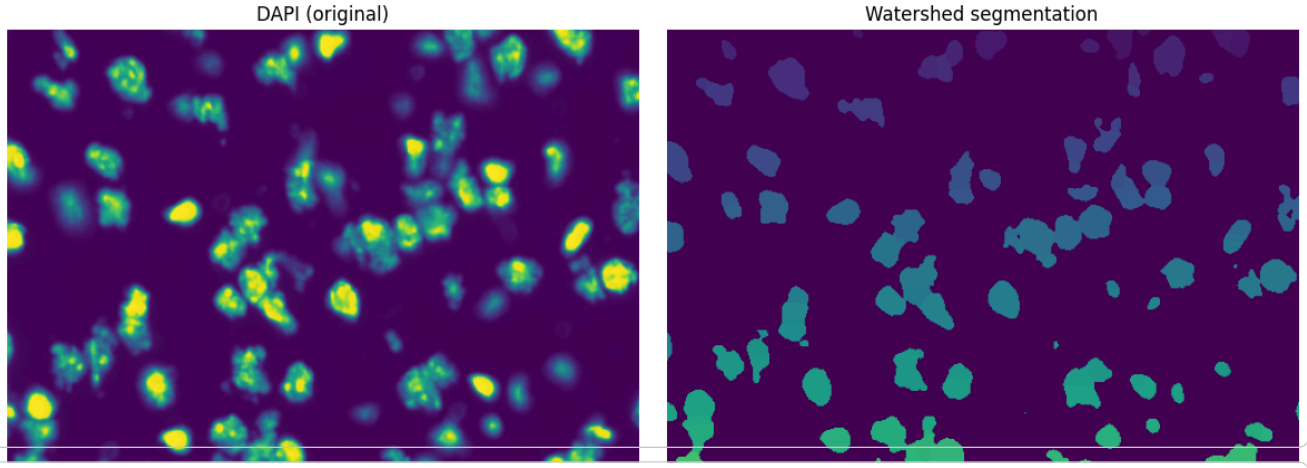



```
sq.im.process(
    img=img,
    layer="image",
    method="smooth",
)

sq.im.segment(
    img=img,
    layer="image_smooth",
    method="watershed",
    channel=0,
    chunks=1000
)

print("Segmentation complete. Layers:", list(img.data.keys()))
```

```
Segmentation complete. Layers: ['image', 'image_smooth', 'segmented_watershed']
```

```
fig, ax = plt.subplots(1, 2, figsize=(12, 6))
img_crop = img.crop_corner(2000, 2000, size=500)
img_crop.show(layer="image", channel=0, ax=ax[0])
ax[0].set_title("DAPI (original)")
img_crop.show(layer="segmented_watershed", channel=0, ax=ax[1])
ax[1].set_title("Watershed segmentation")
plt.tight_layout()
save_fig("segmentation_DAPI_vs_watershed", fig)
plt.show()
```

```
Saved: 03_segmentation_DAPI_vs_watershed.png
```



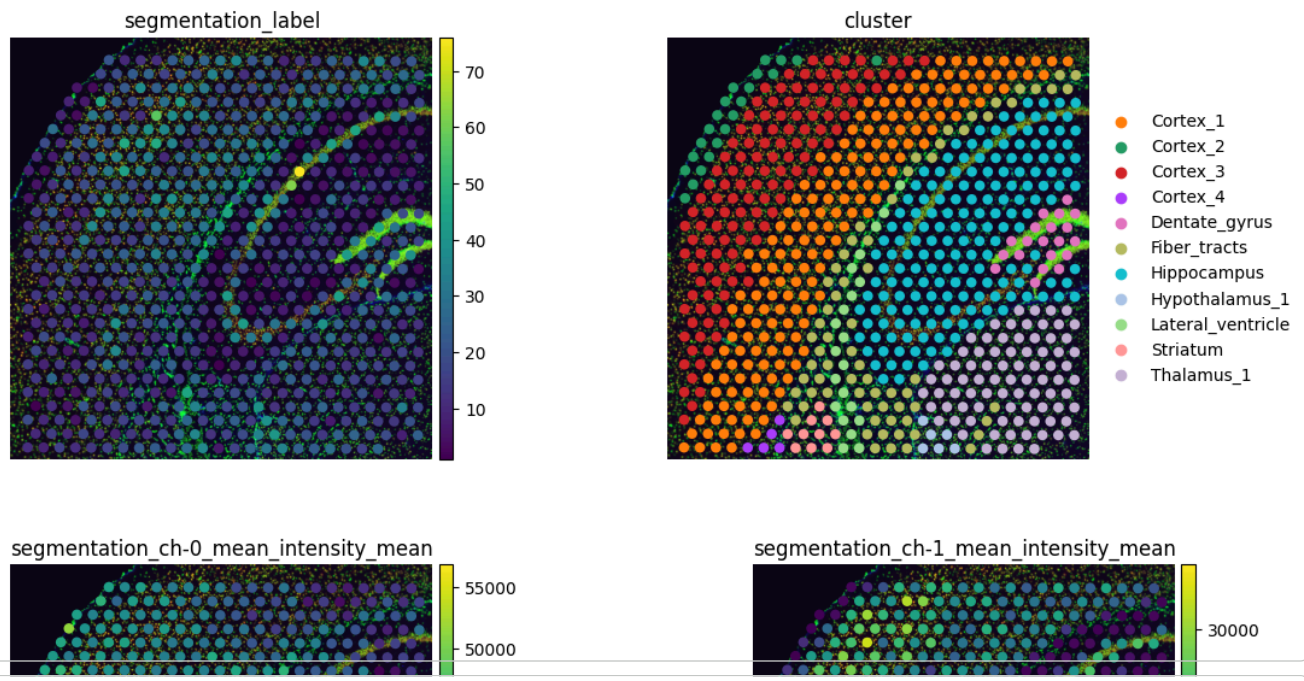

```
features_kwargs = {"segmentation": {"label_layer": "segmented_watershed"}}

sq.im.calculate_image_features(
    adata,
    img,
    features="segmentation",
    layer="image",
    key_added="features_segmentation",
    n_jobs=1,
    features_kwargs=features_kwargs,
```

```
)

print("Segmentation features shape:", adata.obsm["features_segmentation"].shape)
print("Columns:", list(adata.obsm["features_segmentation"].columns))
```

```
100%                                        704/704  [01:48<00:00,   6.83/s]
Segmentation features shape: (704, 9)
Columns: ['segmentation_label', 'segmentation_area_mean', 'segmentation_area_std', 'segmentation_ch-0_mean_intensity_mean', 'seg
```

```
sq.pl.spatial_scatter(
    sq.pl.extract(adata, "features_segmentation"),
    color=[
        "segmentation_label",
        "cluster",
        "segmentation_ch-0_mean_intensity_mean",
        "segmentation_ch-1_mean_intensity_mean",
    ],
    frameon=False,
    ncols=2,
)
save_fig("segmentation_features_spatial")
plt.show()
```

✅ Saved: 04_segmentation_features_spatial.png




```
params = {
    "features_orig": {
        "features": ["summary", "texture", "histogram"],
        "scale": 1.0,
        "mask_circle": True,
    },
    "features_context": {
        "features": ["summary", "histogram"],
        "scale": 1.0,
    },
    "features_lowres": {
        "features": ["summary", "histogram"],
        "scale": 0.25,
    },
}

for feature_name, cur_params in params.items():
    print(f"Extracting: {feature_name} ...")
    sq.im.calculate_image_features(
        adata,
        img,
        layer="image",
        key_added=feature_name,
        n_jobs=1,
```

```
        **cur_params
    )
    print(f"  -> shape: {adata.obsm[feature_name].shape}")

# Combine all features into one dataframe
adata.obsm["features"] = pd.concat(
    [adata.obsm[f] for f in params.keys()], axis="columns"
)

# Remove duplicate column names
adata.obsm["features"].columns = ad.utils.make_index_unique(
    adata.obsm["features"].columns
)

print("\nCombined features shape:", adata.obsm["features"].shape)
```

```
Extracting: features_orig ...
100%                                          704/704  [02:59<00:00,   3.47/s]
  -> shape: (704, 105)
Extracting: features_context ...
100%                                          704/704  [01:40<00:00,   7.17/s]
  -> shape: (704, 45)
Extracting: features_lowres ...
100%                                          704/704  [01:48<00:00,   6.76/s]
  -> shape: (704, 45)

Combined features shape: (704, 195)
```

```
def cluster_features(features: pd.DataFrame, like=None) -> pd.Series:
    """Leiden clustering of image features. Filter columns with `like`."""
    if like is not None:
        features = features.filter(like=like)
    tmp = ad.AnnData(features)
    sc.pp.scale(tmp)
    sc.pp.pca(tmp, n_comps=min(10, features.shape[1] - 1))
    sc.pp.neighbors(tmp)
    sc.tl.leiden(tmp, flavor="igraph", n_iterations=2, directed=False)  # ← fixed
    return tmp.obs["leiden"]
```

```
!pip install squidpy scanpy anndata pandas numpy matplotlib leidenalg igraph -q
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.7/2.7 MB 27.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.7/5.7 MB 43.9 MB/s eta 0:00:00
```

```
adata.obs["features_summary_cluster"]   = cluster_features(adata.obsm["features"], like="summary")
adata.obs["features_histogram_cluster"] = cluster_features(adata.obsm["features"], like="histogram")
adata.obs["features_texture_cluster"]   = cluster_features(adata.obsm["features"], like="texture")

print("Summary clusters:",   adata.obs["features_summary_cluster"].unique().tolist())
print("Histogram clusters:", adata.obs["features_histogram_cluster"].unique().tolist())
print("Texture clusters:",   adata.obs["features_texture_cluster"].unique().tolist())
```

```
Summary clusters: ['0', '2', '1', '3', '4', '5', '6', '7', '8', '9']
Histogram clusters: ['0', '1', '2', '3', '6', '7', '8', '9', '10', '11', '12', '4', '5']
Texture clusters: ['0', '2', '3', '1', '4', '6', '7', '5', '8', '9']
```

```
sc.set_figure_params(facecolor="white", figsize=(8, 8))
sq.pl.spatial_scatter(
    adata,
    color=[
        "features_summary_cluster",
        "features_histogram_cluster",
        "features_texture_cluster",
        "cluster",
    ],
    ncols=2,
)
save_fig("feature_clusters_vs_gene_clusters")
plt.show()
```

```
import shutil
from google.colab import files

saved = sorted(os.listdir(OUTPUT_DIR))
print(f"\n📁 {len(saved)} plot(s) saved:")
for f in saved:
    print(f"   • {f}")

shutil.make_archive("/content/squidpy_visium_fluo_plots", "zip", OUTPUT_DIR)
files.download("/content/squidpy_visium_fluo_plots.zip")
print("✅ Download started!")
```

```
📁 5 plot(s) saved:
   • 01_spatial_scatter_clusters.png
   • 02_fluorescence_channels_DAPI_NEUN_GFAP.png
   • 03_segmentation_DAPI_vs_watershed.png
   • 04_segmentation_features_spatial.png
   • 05_feature_clusters_vs_gene_clusters.png
✅ Download started!
```